

Белорусский государственный технологический университет

Факультет информационных технологий

Кафедра информационных систем и технологий

Лабораторная работа № 9

По дисциплине «Информационная безопасность»

На тему «Исследование асимметричных шифров»

Выполнила:

Студентка 3 курса 9 группы

Фамилия Имя Отчество

Преподаватель: асс. Фамилия Имя
Отчество

Минск 2026

Оглавление

Введение.....	3
1 Ход работы.....	4
1.1 Теоретическое обоснование алгоритма	4
1.2 Генерация ключей	4
1.3 Процесс зашифрования и расшифрования сообщения	7
1.4 Анализ времени выполнения операций	9
Заключение	11

Введение

Асимметричная криптография оперирует парой ключей: открытым (для шифрования) и закрытым (для расшифрования). Данное решение, предложенное в 1976 году У. Диффи и М. Хеллманом, позволило решить проблему безопасного обмена ключами по незащищенным каналам связи.

Одним из первых практических примеров таких систем стал ранцевый алгоритм Меркла-Хеллмана. Его безопасность базируется на NP-полной задаче о сумме подмножеств (задаче об укладке ранца). Злоумышленник, имея только открытый ключ, сталкивается с вычислительно сложной задачей, тогда как легальный получатель, зная «секретную лазейку» (параметры n и a^{-1}), легко восстанавливает исходное сообщение.

Цель работы: изучение и приобретение практических навыков разработки приложений для реализации асимметричных шифров на примере ранцевого алгоритма Меркла-Хеллмана.

Задачи:

1. Закрепить теоретические знания об асимметричных шифрах и задаче укладки ранца.
2. Разработать приложение, реализующее генерацию ключей, шифрование и расшифрование сообщения (ФИО).
3. Проанализировать время выполнения операций при увеличении размера ключа.
4. Сравнить эффективность разных типов кодировки (UTF-8, ASCII, Base64).

1 Ход работы

Для выполнения поставленных задач разработано веб-приложение с использованием HTML, CSS и JavaScript.

1.1 Теоретическое обоснование алгоритма

Ранцевый алгоритм Меркла-Хеллмана основан на следующих понятиях.

Определение 1. Сверхвозрастающая последовательность – последовательность, в которой каждый следующий член больше суммы всех предыдущих. Для такой последовательности задача укладки ранца решается за линейное время.

Определение 2. Открытый ключ получается преобразованием закрытого ключа.

Определение 3. Для расшифрования используется обратный множитель a^{-1} , такой что $a \times a^{-1} \equiv 1 \pmod{n}$, который вычисляется расширенным алгоритмом Евклида.

Основные этапы алгоритма представлены в таблице 1.1.

Таблица 1.1 –Основные этапы ранцевого алгоритма

Этап	Действие	Описание
1	Генерация закрытого ключа	Создание сверхвозрастающей последовательности, где каждый следующий элемент больше суммы всех предыдущих
2	Выбор параметров	Выбор модуля n (больше суммы всех элементов ключа) и множителя a (взаимно простого с n)
3	Вычисление открытого ключа	Преобразование каждого элемента закрытого ключа по формуле: $e_i = (d_i \times a) \pmod{n}$
4	Шифрование блока	Суммирование элементов открытого ключа, соответствующих единичным битам сообщения
5	Расшифрование	Снятие маскировки с помощью обратного множителя и решение задачи укладки ранца

Данные таблицы 1.1 отражают полный цикл криптографического преобразования: от генерации ключей до восстановления исходного сообщения.

1.2 Генерация ключей

Первый этап работы алгоритма – генерация закрытого ключа, который представляет собой сверхвозрастающую последовательность. В разработанном приложении реализована генерация такой последовательности с заданной длиной z (размер последовательности). Для обеспечения криптостойкости старшие члены последовательности имеют длину не менее 40 бит.

Код функции генерации сверхвозрастающей последовательности представлен в листинге 1.1.

```
generateSuperincreasingSequence(z, minBitLength = 40) {
  const sequence = [];
  let sum = 0n;

  for (let i = 0; i < z; i++) {
    let minValue;
    if (i === 0) {
      minValue = (1n << BigInt(minBitLength)) + 1n;
    } else {
      minValue = sum + 1n;
    }
    const maxValue = minValue * 2n;
    const range = maxValue - minValue;
    let randomOffset = 0n;
    if (range > 0n) {
      const maxOffset = range > 10000n ? 10000n : range;
      randomOffset = BigInt(Math.floor(Math.random() *
Number(maxOffset)));
    }
    const value = minValue + randomOffset;
    sequence.push(value);
    sum += value;
  }
  return sequence;
}
```

Листинг 1.1 – Функция генерации сверхвозрастающей последовательности

В листинге 1.1 первый элемент выбирается не менее 2^{40} , а каждый следующий формируется с учетом условия сверхвозрастания, что обеспечивает криптостойкость генерируемой последовательности.

После генерации закрытого ключа вычисляются параметры n (модуль) и a (множитель). Модуль n выбирается больше суммы всех элементов последовательности, а множитель a – взаимно простой с n . Открытый ключ вычисляется по формуле: каждый элемент закрытого ключа умножается на a , затем берется остаток от деления на n .

Код функции генерации ключевой пары представлен в листинге 1.2.

```
generateKeys(z) {
  this.z = z;
  this.privateKey = this.generateSuperincreasingSequence(z);
  const sum = this.privateKey.reduce((a, b) => a + b, 0n);
  this.n = sum + 1n + BigInt(Math.floor(Math.random() *
10000));
}
```

```

let possibleA = [];
for (let i = 3; i < 500; i++) {
    if (this.gcdExtended(i, this.n).gcd === 1n) {
        possibleA.push(i); }
    }
this.a = possibleA[Math.floor(Math.random() *
possibleA.length)];
this.aInverse = this.modInverse(this.a, this.n);
this.publicKey = this.privateKey.map(d => Number((BigInt(d)
* BigInt(this.a)) % BigInt(this.n)));
return { privateKey, publicKey, n: this.n, a: this.a,
aInverse: this.aInverse };
}

```

Листинг 1.2 – Функция генерации ключевой пары

В листинге 1.2 показан полный процесс генерации ключевой пары. Функция *modInverse* реализует расширенный алгоритм Евклида для нахождения обратного элемента a^{-1} , который необходим для расшифрования.

Результат генерации ключей для кодировки ASCII ($z=8$) показан на рисунке 1.2.

Рисунок 1.2 – Результат генерации ключей

Как видно из рисунка 1.2, тайный ключ является сверхвозрастающей последовательностью, а открытый ключ после преобразования утратил это свойство. Значения параметров n и a также отображаются в интерфейсе.

Результат генерации ключей для кодировки Base64 ($z=6$) показан на рисунке 1.3.

Рисунок 1.3 – Результат генерации ключей

Как показано на рисунке 1.3, при использовании кодировки Base64 размер последовательности z уменьшается до 6, так как Base64 кодирует 6 бит

на символ. Тайный ключ сохраняет свойство сверхвозрастания, а открытый ключ его утрачивает.

1.3 Процесс зашифрования и расшифрования сообщения

Для шифрования исходное сообщение преобразуется в бинарную строку, разбивается на блоки длиной z , и для каждого блока вычисляется сумма элементов открытого ключа, соответствующих единичным битам. Расшифрование выполняется в обратном порядке: с помощью обратного множителя a^{-1} снимается маскировка, затем полученное число раскладывается с использованием закрытой сверхвозрастающей последовательности.

Код функций шифрования и расшифрования представлен в листинге 1.3.

```

encrypt(message, useUtf8 = true) {
  let bits = useUtf8 ? this.stringToUtf8Bits(message) :
this.stringToAsciiBits(message);
  const originalBitLength = bits.length;
  const blocks = [];
  for (let i = 0; i < bits.length; i += this.z) {
    let block = bits.substring(i, i + this.z);
    if (block.length < this.z) block = block.padEnd(this.z,
'0');
    blocks.push(block);
  }
  const encrypted = blocks.map(block =>
this.encryptBlock(block, this.publicKey));
  return JSON.stringify({ originalBitLength, data:
encrypted.map(v => v.toString()) });
}
decryptBlock(valueStr) {
  const value = BigInt(valueStr);
  const s = (value * BigInt(this.aInverse)) % BigInt(this.n);
  let remaining = s;
  const bits = new Array(this.z).fill('0');
  const pkBig = this.privateKey.map(k => BigInt(k));
  for (let i = this.z - 1; i >= 0; i--) {
    if (remaining >= pkBig[i]) {
      bits[i] = '1';
      remaining -= pkBig[i];
    }
  }
  return bits.join('');
}

```

Листинг 1.3 – Функции шифрования и расшифрования

Результат шифрования и расшифрования сообщения «Фамилия Имя Отчество» для кодировки ASCII показан на рисунке 1.4.

Зашифрование сообщения

Использовать ФИО

Зашифровать

Зашифрованное сообщение

```
{
  "originalBitLength": 272,
  "data": [
    "213305258533392", "376032981451391", "156130653042194", "208907211932336", "261683770660459", "305664236349243", "411217353964640", "384829074577115", "226499398144342", "42809540338673", "226499398144342", "57174605299734", "292470096764124", "226499398144342", "270479863833126", "332052515689833", "226499398144342", "270479863833126", "314460329547741", "235295491319869", "226499398144342", "57174605299734", "248489631049509", "208907211932336", "226499398144342", "120946280528937", "314460329547741", "34964470263858", "314460329547741", "270479863833126", "420013447093232", "305664236349243", "279275956961710", "226499398144342"]
  ]
}
```

Время зашифрования: 0.200 мс

Расшифрование сообщения

Расшифровать

Расшифрованное сообщение

Время расшифрования: 0.100 мс

Рисунок 1.4 – Результат шифрования и расшифрования (ASCII)

Как видно из рисунка 1.4, исходное текстовое сообщение было преобразовано в зашифрованный вид. При нажатии кнопки «Расшифровать» сообщение было полностью восстановлено, что подтверждает корректность реализации алгоритма.

Результат шифрования и расшифрования сообщения для кодировки Base64 показан на рисунке 1.5.

Зашифрование сообщения

Использовать ФИО

Зашифровать

Зашифрованное сообщение

```
{
  "originalBitLength": 512,
  "data": [
    "57174605017375", "48378511908241", "52776558456711", "50577535220122", "15393162817004", "123145303085497", "17592186148415", "65970698037126", "57174605017375", "83562884216537", "125344326322086", "15393162911826", "107752140173671", "107752140268493", "17592186148415", "48378511877245", "57174605017375", "46179488671652", "79164837745256", "15393162911826", "40159954013790", "107752140173671", "17592186148415", "30786325728830", "57174605017375", "46179488671652", "87960930777201", "15393162817004", "57174605017375", "65970698068122", "83562884185541", "15393162911826", "72567767865375", "92358977356667", "43980465436960", "0", "57174605017375", "83562884216537", "68169721273715", "50577535220122", "15393162817004", "107752140173671", "17592186148415", "30786325728830", "30786325759826", "92358977356667", "17592186148415", "32985349060241", "57174605017375", "83562884216537", "125344326322086", "15393162911826", "72567767865375", "92358977356667", "17592186148415", "57174605017375", "57174605017375", "83562884216537", "68169721273715", "15393162911826", "125344326322086", "92358977356667", "17592186148415", "61572651488656", "57174605017375", "46179488671652", "52776558456711", "15393162911826", "125344326322086", "107752140173671", "17592186148415", "48378511877245", "57174605017375", "83562884216537", "109951163474086", "15393162911826", "72567767865375", "0"]
  ]
}
```

Время зашифрования: 0.200 мс

Расшифрование сообщения

Расшифровать

Расшифрованное сообщение

Время расшифрования: 0.100 мс

Рисунок 1.5 – Результат шифрования и расшифрования (Base64)

Как показано на рисунке 1.5, при использовании кодировки Base64 шифротекст имеет аналогичную структуру. Расшифрованное сообщение полностью совпадает с исходным, что подтверждает корректность работы алгоритма для обоих типов кодировки.

1.4 Анализ времени выполнения операций

Для оценки производительности проведен анализ времени выполнения операций генерации ключей, шифрования и расшифрования при различных значениях размера ключевой последовательности z . Тестирование выполнялось для кодировок ASCII (с базовым размером $z=8$) и Base64 (с базовым размером $z=6$). В качестве тестового сообщения использовалось собственное ФИО. Каждый замер выполнялся многократно для получения усредненных значений. Полученные результаты позволяют оценить практическую производительность алгоритма и выявить зависимость времени выполнения от выбранных параметров. Сравнение двух типов кодировки дает представление о влиянии размера блока на эффективность шифрования.

График зависимости времени выполнения операций от размера ключа z для кодировки ASCII показан на рисунке 1.6.

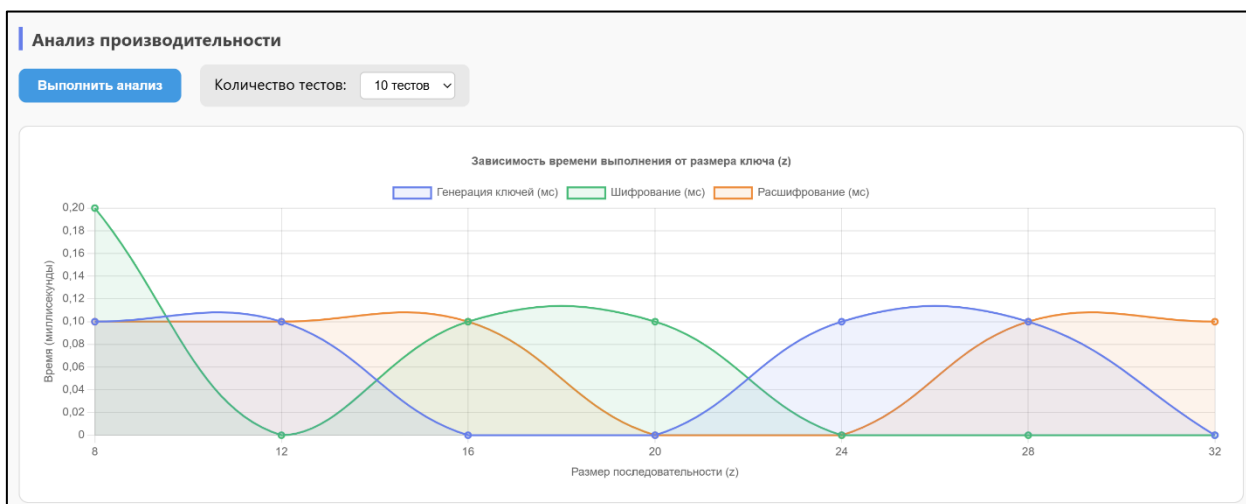


Рисунок 1.6 – График зависимости времени выполнения от z (ASCII)

Как видно из рисунка 1.6, зависимости времени шифрования и расшифрования от размера ключа z имеют линейный характер, что соответствует теоретической сложности алгоритма $O(z)$. Время генерации ключей также растет с увеличением z , но несколько быстрее, что объясняется необходимостью выполнения более сложных операций: поиск взаимно простого множителя a и вычисление обратного элемента a^{-1} по расширенному алгоритму Евклида.

График зависимости времени выполнения операций от размера ключа z для кодировки Base64 показан на рисунке 1.7.



Рисунок 1.7 – График зависимости времени выполнения от z (Base64)

Как показано на рисунке 1.7, для кодировки Base64 также наблюдается линейный рост времени выполнения при увеличении z . При одинаковых значениях z время выполнения операций для Base64 немного ниже, чем для ASCII. Это объясняется тем, что Base64 кодирует 6 бит на символ, поэтому при шифровании того же сообщения требуется обработать меньше блоков, чем при использовании ASCII (8 бит на символ).

Таким образом, анализ производительности подтверждает, что разработанное приложение работает эффективно, а время выполнения операций предсказуемо растет с увеличением размера ключевой последовательности.

Заключение

В ходе выполнения лабораторной работы была достигнута поставленная цель: изучены принципы построения асимметричных шифров на примере ранцевого алгоритма Меркла-Хеллмана и приобретены практические навыки разработки приложений для их реализации.

В соответствии с заданием было разработано веб-приложение, реализующее следующие операции:

- генерация сверхвозрастающей последовательности (тайного ключа) с возможностью задания размера z ;
- вычисление нормальной последовательности (открытого ключа) на основе параметров n и a с использованием расширенного алгоритма Евклида для нахождения обратного элемента;
- зашифрование сообщения (фамилия, имя, отчество) с поддержкой трех типов кодировки: UTF-8, ASCII и Base64;
- расшифрование сообщения с использованием тайного ключа;
- оценка времени выполнения операций шифрования и расшифрования;
- анализ производительности при увеличении размера ключевой последовательности с визуализацией результатов в виде графиков.

Экспериментально подтверждено, что при увеличении размера ключевой последовательности z от 8 до 32 время выполнения операций возрастает линейно, что соответствует теоретической сложности алгоритма $O(z)$. Время генерации ключей растет несколько быстрее времени шифрования и расшифрования из-за необходимости выполнения более сложных математических операций (поиск взаимно простого числа, расширенный алгоритм Евклида). При использовании кодировки Base64 время выполнения операций оказалось ниже по сравнению с ASCII, так как Base64 кодирует 6 бит на символ, что требует обработки меньшего количества блоков.

Разработанное приложение корректно обрабатывает кириллические символы благодаря использованию UTF-8 кодировки. Расшифрованное сообщение полностью совпадает с исходным, что подтверждает корректность реализации всех этапов криптографического преобразования.

Все поставленные задачи выполнены. Разработанное приложение может быть использовано в учебном процессе для демонстрации принципов работы асимметричных криптосистем на основе задачи об укладке ранца, а также для проведения сравнительного анализа производительности при различных параметрах ключа и типах кодировки.